

0.database_strategy_telecom_banking

GIẢI PHÁP DATABASE TOÀN DIỆN CHO DOANH NGHIỆP VIỄN THÔNG

(Mở rộng: Ngân hàng · Tài chính · Thuế · Bảo hiểm)

Tác giả: Trần Văn Bình — Chuyên gia hàng đầu về System/Database tại Việt Nam

Hotline/Zalo: 090.29.12.888 · Website: www.tranvanbinh.vn

KIẾN TRÚC TỔNG QUAN — Khung Quyết định Chọn Database

BÀI TOÁN DOANH NGHIỆP VIỄN THÔNG/TÀI CHÍNH – Database Decision Map

CÂU HỎI 1: Dữ liệu có cần ACID tuyệt đối không?

(tiền, cước phí, giao dịch tài chính, hợp đồng)

CÓ → RDBMS (Oracle/PostgreSQL/SQL Server/MySQL)

KHÔNG → tiếp câu hỏi 2

Nhóm A

CÂU HỎI 2: Cấu trúc dữ liệu có cố định (schema) không?

CỐ ĐỊNH, quan hệ phức tạp → RDBMS

LINH HOẠT, bán cấu trúc (JSON) → NoSQL Document(Mongo)

CHỈ key→value đơn giản, tốc độ cực cao → Key-Value(Redis)

Nhóm B

CÂU HỎI 3: Khối lượng ghi CỰC LỚN, phân tán nhiều node?

(CDR hàng tỷ bản ghi/ngày, IoT/5G telemetry)

CÓ → Wide-Column (Cassandra/HBase) hoặc Time-Series

Nhóm C

CÂU HỎI 4: Bài toán trọng tâm là QUAN HỆ/KẾT NỐI thực thể?

(mạng lưới thuê bao, phát hiện gian lận, quan hệ sở hữu)

CÓ → Graph Database (Neo4j)

Nhóm D

CÂU HỎI 5: Cần lưu trữ/phân tích RAW DATA khối lượng PB,

đa dạng nguồn, batch processing dài hạn?

CÓ → Big Data/Hadoop Ecosystem (HDFS/Hive/Spark)

Nhóm E

MỤC LỤC

PHẦN 1: Tổng quan hệ sinh thái Database — ACID vs BASE, CAP theorem

PHẦN 2: Khi nào dùng loại Database nào (Nhóm A-E)

PHẦN 3: Ánh xạ Loại ứng dụng ↔ Database phù hợp (Viễn thông/Ngân hàng/Thuế/Bảo hiểm)

PHẦN 4: Kiến trúc tham chiếu Polyglot Persistence

PHẦN 5: Hướng dẫn Migration chi tiết (Oracle↔MySQL/PostgreSQL/SQL Server/MongoDB)

PHẦN 6: Case Study thực chiến — CDR/Billing Viễn thông

PHẦN 7: Mở rộng sang Ngân hàng/Tài chính/Thuế/Bảo hiểm

PHẦN 8: Checklist Quyết định & Tổng kết

PHẦN 1: TỔNG QUAN HỆ SINH THÁI DATABASE

1.1 ACID vs BASE

Tiêu chí	ACID (RDBMS)	BASE (đa số NoSQL)
Atomicity	Transaction toàn-hoặc-không	Basically Available — ưu tiên luôn phản hồi
Consistency	Dữ liệu luôn nhất quán ngay	Soft state — trạng thái có thể thay đổi
Isolation	Transaction song song không ảnh hưởng nhau	—
Durability	Commit rồi không mất	Eventual consistency — cuối cùng nhất quán
Phù hợp	Tiền, cược, hợp đồng, giao dịch tài chính	Session, cache, log, activity feed

1.2 CAP Theorem áp dụng thực tế

Hệ thống	Ưu tiên	Ý nghĩa thực tế
Oracle RAC, PostgreSQL (Sync)	CP	Thà từ chối phục vụ còn hơn trả dữ liệu sai — đúng cho core banking

Hệ thống	Ưu tiên	Ý nghĩa thực tế
MongoDB (default), Cassandra	AP	Luôn phản hồi, chấp nhận dữ liệu chưa đồng bộ ngay — đúng cho session/CDR
Redis Sentinel	CP thiên hướng	Failover có kiểm soát

1.3 Bảng tổng quan 6 nhóm công nghệ

Nhóm	Đại diện	Mô hình dữ liệu	Đặc tính nổi bật
RDBMS	Oracle, PostgreSQL, SQL Server, MySQL	Bảng quan hệ, schema cố định	ACID mạnh, JOIN phức tạp
NoSQL Document	MongoDB, Couchbase	JSON/BSON linh hoạt	Schema-less, scale ngang dễ
NoSQL Key-Value	Redis, DynamoDB	Key → Value	Tốc độ cực cao (sub-ms)
NoSQL Wide-Column	Cassandra, HBase	Bảng cực rộng, phân tán	Ghi khối lượng cực lớn
Graph	Neo4j, Amazon Neptune	Node + Edge	Truy vấn quan hệ nhiều bước cực nhanh
Big Data/Hadoop	HDFS, Hive, Spark	File phân tán + Schema-on-read	Lưu trữ PB, xử lý batch

PHẦN 2: KHI NÀO DÙNG LOẠI DATABASE NÀO

Nhóm A: RDBMS — khi cần ACID tuyệt đối

Dùng khi: dữ liệu liên quan trực tiếp TIỀN (billing, thanh toán, sổ dư); nhiều ràng buộc toàn vẹn tham chiếu phức tạp; cần transaction đa bảng rollback tuyệt đối; cần JOIN phức tạp cho báo cáo chính xác.

Tiêu chí	Oracle	PostgreSQL	SQL Server	MySQL
Mission-critical, HA cao nhất	✓ ✓ ✓	✓ ✓ (Patroni)	✓ ✓ (AlwaysOn)	✓
Chi phí license	Cao	Miễn phí	Trung bình-cao	Miễn phí
Hệ sinh thái enterprise	✓ ✓ ✓ (PL/SQL, GoldenGate)	Tương đương qua extension	✓ ✓ (T-SQL)	Hạn chế

Tiêu chí	Oracle	PostgreSQL	SQL Server	MySQL
Phù hợp	Core banking, billing lớn (>10TB), SLA 99.999%	Ứng dụng mới, tối ưu chi phí, JSON native	Môi trường .NET/Windows	Web app vừa/nhỏ

Ví dụ quyết định: Billing/Charging real-time nhà mạng lớn → **Oracle** (RAC+Data Guard) vì cần chục nghìn TPS zero data loss. CRM nội bộ quy mô vừa → **PostgreSQL** để tối ưu chi phí.

Nhóm B: NoSQL Document/Key-Value — khi schema linh hoạt

MongoDB khi: cấu trúc thay đổi thường xuyên theo sản phẩm/gói cước; dữ liệu tự nhiên dạng document lồng nhau; cần scale ngang dễ; không cần JOIN phức tạp.

Redis khi: cần tốc độ dưới mili-giây (session, OTP, rate limiting); dữ liệu tạm thời có TTL; cấu trúc đơn giản.

Ví dụ: Session app di động, cache tra cứu gói cước → **Redis**. Hồ sơ Customer 360 tổng hợp nhiều nguồn → **MongoDB**.

Nhóm C: Wide-Column/Time-Series — khi ghi khối lượng cực lớn

Cassandra/HBase khi: khối lượng ghi cực lớn (CDR hàng tỷ bản ghi/ngày); cần scale ngang tuyến tính, masterless; truy vấn theo partition key đã biết.

Time-Series (InfluxDB/TimescaleDB) khi: dữ liệu bản chất theo dòng thời gian (KPI mạng, IoT/5G); cần hàm tổng hợp theo cửa sổ thời gian.

Ví dụ: CDR 500 triệu bản ghi/ngày → **Cassandra**. KPI trạm 5G mỗi giây → **TimescaleDB**.

Nhóm D: Graph Database — khi trọng tâm là quan hệ

Neo4j khi: bài toán cốt lõi là tìm quan hệ nhiều bước (traversal); phát hiện gian lận (fraud ring); phân tích mạng lưới thuê bao; AML sở hữu doanh nghiệp phức tạp.

Ví dụ: Fraud SIM box, AML rửa tiền → **Neo4j** vì RDBMS self-join 5-6 cấp chậm theo cấp số nhân, Graph traversal gần tuyến tính.

Nhóm E: Big Data/Hadoop — khi lưu trữ/phân tích PB dữ liệu

HDFS/Hive/Spark khi: cần lưu RAW DATA khối lượng PB đa dạng định dạng; xử lý batch dài hạn (churn prediction, training ML); chi phí lưu trữ commodity hardware rẻ; chấp nhận độ trễ phút/giờ.

Ví dụ: Data Lake CDR/log network 5 năm cho phân tích churn → **Hadoop**, kết quả tổng hợp mới đẩy vào RDBMS/DWH cho BI.

PHẦN 3: ẢNH XẠ LOẠI ỨNG DỤNG ↔ DATABASE PHÙ HỢP

3.1 Ngành Viễn thông

Loại ứng dụng	Database khuyến nghị	Lý do
Billing/Charging real-time	Oracle (hoặc PostgreSQL quy mô vừa)	ACID, transaction tài chính, throughput cao
CDR lưu trữ thô	Cassandra hoặc HDFS	Ghi cực lớn, ít update, scale ngang
Customer 360/CRM hợp nhất	MongoDB	Bán cấu trúc, schema thay đổi theo sản phẩm
Session App/Portal	Redis	Tốc độ, TTL tự động
Network Topology/Inventory	Neo4j hoặc PostgreSQL+PostGIS	Quan hệ node-link, dữ liệu địa lý BTS
Fraud Detection (SIM box)	Neo4j + Kafka streaming	Phát hiện pattern quan hệ real-time
KPI mạng 5G/IoT	TimescaleDB/InfluxDB	Time-series tối ưu
Data Lake/phân tích dài hạn	Hadoop (Hive/Spark)	Lưu PB, batch processing
Recommendation gói cước	MongoDB + Redis cache	Linh hoạt, tốc độ real-time
Data Warehouse/BI	Oracle Exadata hoặc PostgreSQL+ClickHouse	Truy vấn phân tích phức tạp

3.2 Ngành Ngân hàng/Tài chính

Loại ứng dụng	Database khuyến nghị	Lý do
Core Banking (tài khoản, sổ cái)	Oracle (bắt buộc mission-critical)	ACID tuyệt đối, RAC HA, kiểm chứng audit
Thanh toán liên ngân hàng	Oracle/PostgreSQL	Transaction chính xác, idempotency
Trading System	PostgreSQL/kdb+	Độ trễ cực thấp, order book
AML	Neo4j	Phát hiện mạng lưới giao dịch/sở hữu bất thường

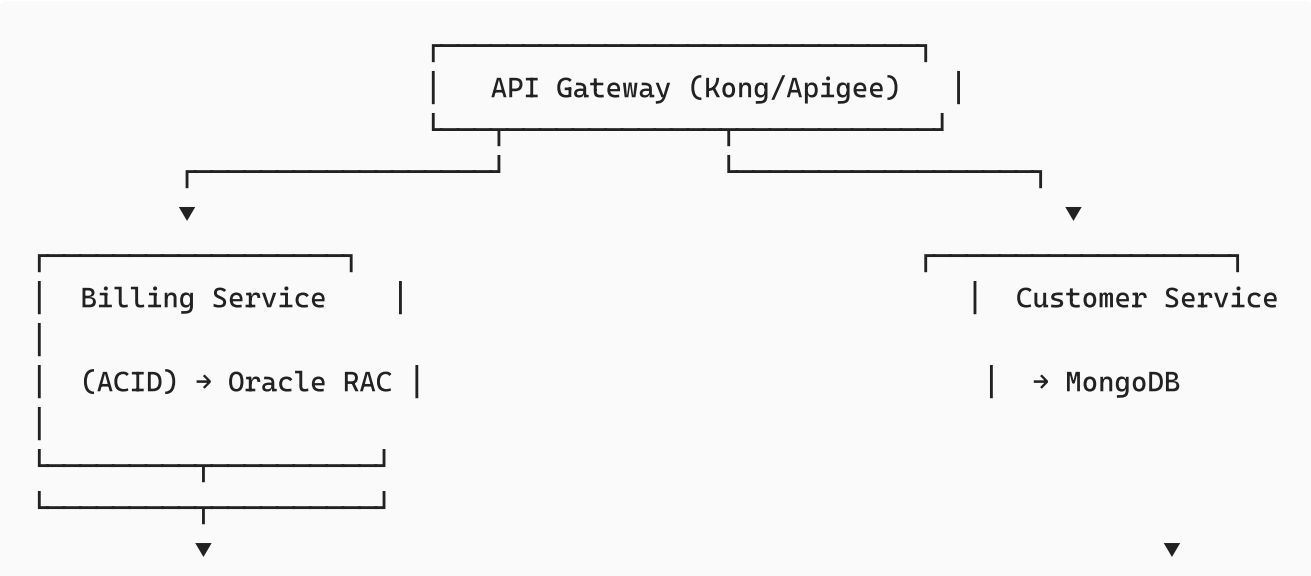
Loại ứng dụng	Database khuyến nghị	Lý do
Mobile Banking session/OTP	Redis	Tốc độ, bảo mật TTL ngắn
Risk Analytics	Hadoop/Spark + DWH	Dữ liệu lớn, training model
KYC hồ sơ	MongoDB	Tài liệu đa dạng định dạng

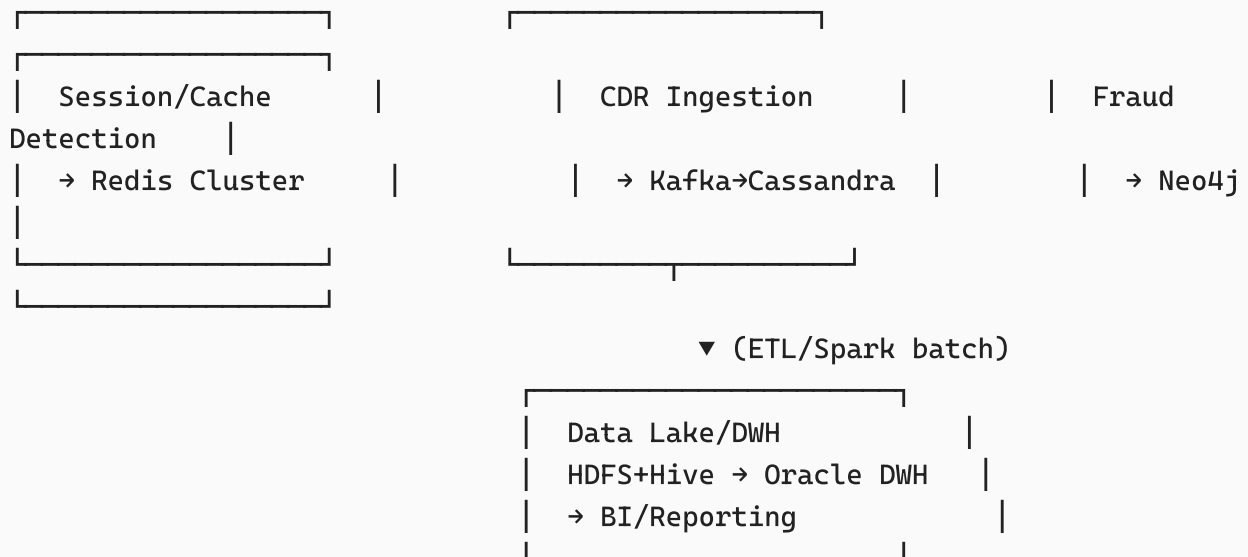
3.3 Ngành Thuế & Bảo hiểm

Loại ứng dụng	Database khuyến nghị	Lý do
Kê khai thuế điện tử	Oracle/PostgreSQL	Ràng buộc nghiệp vụ phức tạp, audit trail
Đối chiếu hóa đơn điện tử	PostgreSQL + Elasticsearch	Chi phí hợp lý, full-text search
Claim Processing bảo hiểm	PostgreSQL/SQL Server	Workflow trạng thái, transaction
Actuarial	Hadoop/Spark	Tính toán thống kê khối lượng lớn
Hồ sơ claim (ảnh, giấy tờ)	MongoDB	Document linh hoạt
Fraud bảo hiểm	Neo4j	Quan hệ người mua-garage-bệnh viện

PHẦN 4: KIẾN TRÚC THAM CHIẾU POLYGLOT PERSISTENCE

Doanh nghiệp lớn không dùng "một database cho tất cả" — kiến trúc thực tế là **polyglot persistence**:





Nguyên tắc: mỗi service SỞ HỮU RIÊNG database (Database per Service), đồng bộ qua event streaming (Kafka), KHÔNG chia sẻ trực tiếp database giữa các service.

PHẦN 5: HƯỚNG DẪN MIGRATION CHI TIẾT

Quy trình chuẩn 6 bước cho MỌI migration

1. ASSESSMENT – Kiểm kê schema, object, khối lượng dữ liệu
2. SCHEMA CONVERT – Chuyển đổi cấu trúc bảng, kiểu dữ liệu, ràng buộc
3. CODE CONVERT – Chuyển PL/SQL/T-SQL, connection string ứng dụng
4. DATA MIGRATION – Full load + CDC đồng bộ liên tục
5. VALIDATION – Đối chiếu row count, checksum, dual-run test
6. CUTOVER – Chuyển traffic thật, có kế hoạch ROLLBACK

5.1 Oracle → MySQL

Khi nào: web/portal quy mô vừa, không cần PL/SQL nặng/partitioning phức tạp, giảm chi phí license.

Công cụ: AWS SCT (Schema Conversion Tool) + AWS DMS, hoặc thủ công.

```

-- Bảng ánh xạ kiểu dữ liệu Oracle → MySQL
-- NUMBER(p,s) → DECIMAL(p,s)          VARCHAR2(n) → VARCHAR(n)
-- CLOB          → LONGTEXT              BLOB          → LONGBLOB
-- DATE          → DATETIME              SEQUENCE      → AUTO_INCREMENT (khác
biệt lớn nhất)
-- ROWNUM        → LIMIT                 NVL()          → IFNULL()
-- CONNECT BY    → WITH RECURSIVE (MySQL 8.0+)
  
```

```
-- Oracle SEQUENCE:
CREATE SEQUENCE seq_customer_id START WITH 1 INCREMENT BY 1;
INSERT INTO customers (id, name) VALUES (seq_customer_id.NEXTVAL, 'Nguyen
Van A');

-- MySQL AUTO_INCREMENT tương đương:
CREATE TABLE customers (id INT AUTO_INCREMENT PRIMARY KEY, name
VARCHAR(200));
INSERT INTO customers (name) VALUES ('Nguyen Van A');
```

```
# Export Oracle ra CSV
sqlplus user/pass@ORCLPDB1 << 'EOF'
SET COLSEP ',' PAGESIZE 0 TRIMSPOOL ON
SPOOL customers.csv
SELECT id||','||name||','||phone FROM customers;
SPOOL OFF
EOF

# Import vào MySQL
mysql -u root -p mydb << 'EOF'
LOAD DATA LOCAL INFILE 'customers.csv' INTO TABLE customers
FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n' (id, name, phone);
EOF

# Khối lượng lớn + CDC liên tục: dùng AWS DMS
aws dms create-replication-task \
  --replication-task-identifier oracle-to-mysql-cdc \
  --source-endpoint-arn arn:aws:dms:...:endpoint:oracle-source \
  --target-endpoint-arn arn:aws:dms:...:endpoint:mysql-target \
  --migration-type full-load-and-cdc --table-mappings file://table-
mappings.json
```

```
-- Chuyển PL/SQL Procedure sang MySQL Stored Procedure
-- Oracle:
CREATE OR REPLACE PROCEDURE update_balance(p_cust_id NUMBER, p_amount
NUMBER) IS
BEGIN
  UPDATE accounts SET balance = balance + p_amount WHERE customer_id =
p_cust_id;
  IF SQL%ROWCOUNT = 0 THEN RAISE_APPLICATION_ERROR(-20001, 'Customer not
found'); END IF;
  COMMIT;
END;
/

-- MySQL:
DELIMITER $$
CREATE PROCEDURE update_balance(IN p_cust_id INT, IN p_amount DECIMAL(15,2))
```

```
BEGIN
  UPDATE accounts SET balance = balance + p_amount WHERE customer_id =
p_cust_id;
  IF ROW_COUNT() = 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Customer not found'; END IF;
END$$
DELIMITER ;
```

5.2 Oracle → PostgreSQL (dùng ora2pg)

Khi nào: hướng PHỔ BIẾN NHẤT hiện nay — giảm chi phí license mà vẫn giữ đặc tính RDBMS enterprise (PL/pgSQL tương đồng PL/SQL, JSON native, partitioning).

```
# Cài đặt ora2pg
dnf install -y perl perl-DBI perl-DBD-Oracle ora2pg
ora2pg --project_base /migration --init_project telco_migration
cd /migration/telco_migration
```

```
# config/ora2pg.conf
ORACLE_DSN      dbi:Oracle:host=oracle-host;service_name=ORCLPDB1;port=1521
ORACLE_USER     migration_user
ORACLE_PWD      StrongP@ssw0rd
SCHEMA          TELCO_BILLING
PG_SCHEMA       telco_billing
PARALLEL_TABLES 4
JOBS            4
```

```
# Chạy TÁCH RIÊNG từng giai đoạn (không gộp một lệnh)
ora2pg -t TABLE      -c config/ora2pg.conf -o schema.sql      # 1. Cấu trúc
bảng trước
ora2pg -t SEQUENCE    -c config/ora2pg.conf -o sequences.sql
ora2pg -t COPY        -c config/ora2pg.conf -o data.sql        # 2. QUAN TRỌNG
NHẤT - dữ liệu
ora2pg -t INDEX       -c config/ora2pg.conf -o indexes.sql     # 3. Index SAU
khi có dữ liệu (nhanh hơn)
ora2pg -t FKEY        -c config/ora2pg.conf -o fkeys.sql
ora2pg -t CHECK       -c config/ora2pg.conf -o checks.sql
ora2pg -t VIEW        -c config/ora2pg.conf -o views.sql
ora2pg -t PACKAGE     -c config/ora2pg.conf -o packages.sql    #
PL/SQL→PL/pgSQL, review kỹ (~70-80% tự động)
ora2pg -t FUNCTION    -c config/ora2pg.conf -o functions.sql
ora2pg -t TRIGGER     -c config/ora2pg.conf -o triggers.sql

# Áp dụng vào PostgreSQL ĐÚNG THỨ TỰ
psql -h pg-host -U postgres -d telco_db -f schema.sql
psql -h pg-host -U postgres -d telco_db -f sequences.sql
psql -h pg-host -U postgres -d telco_db -f data.sql
```

```
psql -h pg-host -U postgres -d telco_db -f indexes.sql
psql -h pg-host -U postgres -d telco_db -f fkeys.sql
psql -h pg-host -U postgres -d telco_db -f functions.sql
```

```
-- Ví dụ chuyển PL/SQL Package sang PL/pgSQL (package → schema + function riêng lẻ)
```

```
-- Oracle:
```

```
CREATE OR REPLACE PACKAGE BODY billing_pkg AS
    PROCEDURE charge_customer(p_id IN customers.id%TYPE, p_amount IN NUMBER)
IS
    insufficient_balance EXCEPTION;
BEGIN
    UPDATE accounts SET balance = balance - p_amount WHERE customer_id =
p_id;
    IF (SELECT balance FROM accounts WHERE customer_id = p_id) < 0 THEN
        RAISE insufficient_balance;
    END IF;
    COMMIT;
EXCEPTION
    WHEN insufficient_balance THEN
        ROLLBACK; RAISE_APPLICATION_ERROR(-20001, 'Số dư không đủ');
END;
END billing_pkg;
/
```

```
-- PostgreSQL:
```

```
CREATE SCHEMA IF NOT EXISTS billing_pkg;
CREATE OR REPLACE FUNCTION billing_pkg.charge_customer(p_id
customers.id%TYPE, p_amount NUMERIC)
RETURNS VOID AS $$
BEGIN
    UPDATE accounts SET balance = balance - p_amount WHERE customer_id = p_id;
    IF (SELECT balance FROM accounts WHERE customer_id = p_id) < 0 THEN
        RAISE EXCEPTION 'Số dư không đủ' USING ERRCODE = '20001';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
# Đồng bộ liên tục (CDC) trong giai đoạn chuyển đổi song song
ora2pg -t COPY --cdc_type synchronous -c config/ora2pg.conf
ora2pg -t TEST -c config/ora2pg.conf # đối chiếu cơ bản
```

5.3 Oracle → SQL Server (dùng SSMA)

Khi nào: hạ tầng .NET/Windows-centric, tích hợp Power BI/Azure/Active Directory.

Quy trình SSMA (SQL Server Migration Assistant for Oracle — công cụ chính thức miễn phí):

1. Cài SSMA for Oracle (Windows) → New Project, chọn SQL Server version đích
2. Connect to Oracle (TNS/connection string) + Connect to SQL Server đích
3. Oracle Metadata Explorer: chọn schema → kéo sang panel phải
4. "Convert Schema" → xem "Conversion Report" (% tự động, cảnh báo cần sửa tay)
5. "Synchronize with Database" → tạo schema thật trên SQL Server
6. "Migrate Data" → multi-thread, resumable

```
-- Ánh xạ kiểu dữ liệu Oracle → SQL Server
-- NUMBER → DECIMAL/NUMERIC/INT      VARCHAR2(4000) →
VARCHAR(4000)/NVARCHAR(MAX)
-- CLOB → VARCHAR(MAX)/NVARCHAR(MAX)  DATE → DATETIME2
-- SEQUENCE → IDENTITY hoặc SEQUENCE object (SQL Server 2012+)
-- PACKAGE → Schema + nhiều Stored Procedure riêng lẻ

-- Cursor Oracle → T-SQL
-- Oracle:
DECLARE
    CURSOR c_customers IS SELECT id, name FROM customers WHERE status =
'ACTIVE';
BEGIN
    FOR rec IN c_customers LOOP
        UPDATE accounts SET last_review = SYSDATE WHERE customer_id = rec.id;
    END LOOP;
    COMMIT;
END;
/

-- SQL Server:
DECLARE @id INT, @name NVARCHAR(200);
DECLARE c_customers CURSOR FOR SELECT id, name FROM customers WHERE status =
'ACTIVE';
OPEN c_customers;
FETCH NEXT FROM c_customers INTO @id, @name;
WHILE @@FETCH_STATUS = 0
BEGIN
    UPDATE accounts SET last_review = GETDATE() WHERE customer_id = @id;
    FETCH NEXT FROM c_customers INTO @id, @name;
END
CLOSE c_customers; DEALLOCATE c_customers;
```

```
# SSMA Console (headless, cho CI/CD) + BCP cho migration dữ liệu thô nhanh
SSMAforOracleConsole.exe -s migration_project.xml -o output.log
bcp TelcoDB.dbo.customers in customers.dat -c -t, -S sqlserver-host -U sa -P
Password123
```

5.4 Oracle → MongoDB (thiết kế lại theo mô hình Document)

Khi nào: hướng khác biệt nhất — KHÔNG chỉ convert cú pháp mà cần THIẾT KẾ LẠI mô hình dữ liệu từ quan hệ (nhiều bảng, JOIN) sang document (nhúng lồng nhau).

```
-- Oracle: mô hình chuẩn hóa (normalized) - 3 bảng, cần JOIN
CREATE TABLE customers (id NUMBER PRIMARY KEY, name VARCHAR2(200), email
VARCHAR2(200));
CREATE TABLE addresses (id NUMBER PRIMARY KEY, customer_id NUMBER REFERENCES
customers(id),
    address_type VARCHAR2(20), full_address VARCHAR2(500));
CREATE TABLE subscriptions (id NUMBER PRIMARY KEY, customer_id NUMBER
REFERENCES customers(id),
    plan_name VARCHAR2(100), start_date DATE, status VARCHAR2(20));

SELECT c.name, a.full_address, s.plan_name FROM customers c
JOIN addresses a ON a.customer_id = c.id
JOIN subscriptions s ON s.customer_id = c.id WHERE c.id = 1001;
```

```
// MongoDB: NHÚNG (embed) dữ liệu liên quan vào MỘT document duy nhất
{
  "_id": ObjectId("..."),
  "customer_id": 1001,
  "name": "Nguyen Van A",
  "email": "nguyenvana@email.com",
  "addresses": [
    { "type": "HOME", "full_address": "123 Le Loi, Q1, HCMC" },
    { "type": "OFFICE", "full_address": "456 Nguyen Hue, Q1, HCMC" }
  ],
  "subscriptions": [
    { "plan_name": "5G Premium", "start_date": ISODate("2025-01-15"),
  "status": "ACTIVE" }
  ]
}

// Truy vấn - KHÔNG cần JOIN, lấy toàn bộ trong MỘT lần đọc
db.customers.findOne({ customer_id: 1001 })

// Nguyên tắc EMBED vs REFERENCE:
// - EMBED khi: dữ liệu con LUÔN đọc CÙNG cha, quan hệ 1-N với N nhỏ
// - REFERENCE khi: dữ liệu con LỚN/thay đổi độc lập, hoặc N-N (gói cước
dùng chung nên REFERENCE)
```

```
# migrate_oracle_to_mongo.py - KHÔNG có công cụ tự động "convert schema" vì
bản chất
# migration này đòi hỏi redesign, không phải convert 1-1
import cx_Oracle
from pymongo import MongoClient
```

```

from datetime import datetime

oracle_conn = cx_Oracle.connect("user/password@oracle-host:1521/ORCLPDB1")
mongo_db = MongoClient("mongodb://mongo-host:27017/")[telco_customer360]

cursor = oracle_conn.cursor()
cursor.execute("SELECT id, name, email FROM customers")
batch = []
for cust_id, name, email in cursor:
    addr_cursor = oracle_conn.cursor()
    addr_cursor.execute("SELECT address_type, full_address FROM addresses
WHERE customer_id = :1", [cust_id])
    addresses = [{"type": t, "full_address": a} for t, a in addr_cursor]

    sub_cursor = oracle_conn.cursor()
    sub_cursor.execute("SELECT plan_name, start_date, status FROM
subscriptions WHERE customer_id = :1", [cust_id])
    subscriptions = [{"plan_name": p, "start_date": d, "status": s} for p,
d, s in sub_cursor]

    doc = {"customer_id": cust_id, "name": name, "email": email,
"addresses": addresses, "subscriptions": subscriptions,
"migrated_at": datetime.utcnow()}
    batch.append(doc)
    if len(batch) >= 1000:
        mongo_db.customers.insert_many(batch)
        batch = []
if batch:
    mongo_db.customers.insert_many(batch)
print("Migration hoàn tất!")

```

```

// Tạo Index SAU khi có dữ liệu (tương tự nguyên tắc RDBMS)
db.customers.createIndex({ "customer_id": 1 }, { unique: true })
db.customers.createIndex({ "email": 1 })
db.customers.createIndex({ "subscriptions.status": 1 }) // index trên
field nhúng trong mảng

```

```

# Validation - đối chiếu số lượng document
sqlplus -s user/pass@ORCLPDB1 <<< "SELECT COUNT(*) FROM customers;"
mongosh --eval "db.customers.countDocuments()" telco_customer360

```

5.5 PostgreSQL/SQL Server → Oracle (khi cần nâng cấp Mission-Critical)

Khi nào (migration NGƯỜI): ứng dụng ban đầu chọn PostgreSQL/SQL Server cho hệ vừa/nhỏ, nhưng sau tăng trưởng trở thành CORE cần Oracle RAC+Data Guard+GoldenGate hoặc SLA khắt khe hơn.

Công cụ: Oracle SQL Developer Migration Workbench (hỗ trợ CẢ PostgreSQL và SQL Server làm nguồn).

```
# View > Migration > Migration Wizard trong SQL Developer, hoặc headless mode:
```

```
$ORACLE_HOME/sqldeveloper/bin/sdcli migration capture \  
-conn "jdbc:postgresql://pg-host:5432/telco_db" \  
-user postgres -password StrongP@ssw0rd -outputPath /migration/capture  
  
$ORACLE_HOME/sqldeveloper/bin/sdcli migration convert \  
-captureFile /migration/capture/model.xml -outputPath /migration/convert
```

```
-- Ánh xạ kiểu dữ liệu PostgreSQL → Oracle  
-- SERIAL/BIGSERIAL → NUMBER+SEQUENCE+TRIGGER (hoặc IDENTITY từ Oracle 12c+)  
-- TEXT → CLOB (hoặc VARCHAR2(4000) nếu chắc chắn ngắn)  
-- JSONB → JSON (Oracle 21c+ native) hoặc CLOB+CHECK constraint  
-- BOOLEAN → NUMBER(1) hoặc CHAR(1) CHECK ('Y'/'N')  
-- ARRAY → không hỗ trợ trực tiếp, cần bảng con hoặc VARRAY
```

```
-- PostgreSQL:
```

```
CREATE TABLE customers (id SERIAL PRIMARY KEY, name TEXT, metadata JSONB);
```

```
-- Oracle 12c+ tương đương:
```

```
CREATE TABLE customers (  
    id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name CLOB,  
    metadata JSON    -- Oracle 21c+ native; 12c-19c dùng CLOB + CHECK (metadata  
IS JSON)  
);
```

```
-- PL/pgSQL Function → PL/SQL Procedure
```

```
-- PostgreSQL:
```

```
CREATE OR REPLACE FUNCTION update_balance(p_cust_id INT, p_amount NUMERIC)  
RETURNS VOID AS $$  
BEGIN  
    UPDATE accounts SET balance = balance + p_amount WHERE customer_id =  
p_cust_id;  
    IF NOT FOUND THEN RAISE EXCEPTION 'Customer not found'; END IF;  
END;  
$$ LANGUAGE plpgsql;
```

```
-- Oracle:
```

```
CREATE OR REPLACE PROCEDURE update_balance(p_cust_id IN NUMBER, p_amount IN  
NUMBER) IS  
BEGIN  
    UPDATE accounts SET balance = balance + p_amount WHERE customer_id =  
p_cust_id;
```

```
IF SQL%ROWCOUNT = 0 THEN RAISE_APPLICATION_ERROR(-20001, 'Customer not  
found'); END IF;  
COMMIT;  
END;  
/
```

```
# Zero-Downtime Migration cho production: Oracle GoldenGate  
# 1. GoldenGate Extract trên PostgreSQL nguồn (GoldenGate for PostgreSQL)  
# 2. GoldenGate Replicat áp dụng liên tục vào Oracle đích  
# 3. Chạy SONG SONG (dual-run) một thời gian để validate  
# 4. Cutover khi đã xác nhận đồng bộ 100%  
  
# Khối lượng vừa/nhỏ, dùng SQL*Loader qua CSV trung gian:  
sqlldr userid=oracle_user/pass@ORCLPDB1 control=customers.ctl  
log=customers.log
```

PHẦN 6: CASE STUDY THỰC CHIẾN — Hệ thống CDR/Billing Viễn thông

Bối cảnh: Nhà mạng vừa (5 triệu thuê bao), ban đầu MỘT Oracle Database cho MỌI thứ (billing+CDR+customer+fraud) — quá tải khi CDR lên 300 triệu bản ghi/ngày, chi phí license lẫn phí cho khối lượng ít cần ACID.

GIAI ĐOẠN 1 (Tháng 1-2): Tách CDR khỏi Oracle

- Kafka nhận CDR real-time từ Network Element (MSC/SGSN)
- Cassandra cluster (6 node) lưu CDR thô
- Oracle giữ Billing, chỉ đọc CDR tổng hợp qua Spark job hàng giờ
- Kết quả: giảm 70% dung lượng Oracle, tăng throughput ghi CDR 10 lần

GIAI ĐOẠN 2 (Tháng 3-4): Tách Customer 360 sang MongoDB

- Script Python migration (Phần 5.4) chuyển Customer profile
- Đồng bộ 2 chiều tạm qua Kafka CDC trong giai đoạn chuyển đổi
- CRM/App di động đọc MongoDB (nhanh hơn Oracle 3-5 lần cho profile)

GIAI ĐOẠN 3 (Tháng 5): Triển khai Neo4j cho Fraud Detection

- Stream CDR từ Kafka vào CẢ Cassandra (lưu trữ) VÀ Neo4j (phân tích)
- Rule phát hiện SIM box: nhiều cuộc gọi ngắn bất thường từ 1 nhóm SIM

GIAI ĐOẠN 4 (Tháng 6): Oracle chỉ còn Billing/Charging lõi

- Downsizing license Oracle (giảm core do dữ liệu giảm mạnh)
- Data Guard+GoldenGate CHỈ áp dụng Oracle Billing (mission-critical thực sự)

KẾT QUẢ: Giảm 40% chi phí license Oracle, tăng khả năng scale CDR 20 lần, phát hiện fraud từ vài giờ xuống real-time (<5 giây)

PHẦN 7: MỞ RỘNG SANG NGÂN HÀNG/TÀI CHÍNH/THUẾ/BẢO HIỂM

Nguyên tắc	Viễn thông	Ngân hàng	Thuế	Bảo hiểm
Core giao dịch tiền → RDBMS	Billing → Oracle	Core Banking → Oracle	Hạch toán → Oracle/PostgreSQL	Phí/bồi thường → PostgreSQL/SQL Server
Khối lượng lớn, ít ACID → NoSQL/BigData	CDR → Cassandra/HDFS	Log giao dịch thẻ → Cassandra	Log tờ khai → HDFS	Log claim → MongoDB
Quan hệ/mạng lưới → Graph	Fraud SIM box → Neo4j	AML → Neo4j	Gian lận hóa đơn chuỗi → Neo4j	Fraud ring → Neo4j
Tốc độ/session → Key-Value	Session App → Redis	Mobile Banking OTP → Redis	Session cổng thuế → Redis	Session Claim App → Redis
Document linh hoạt → NoSQL	Customer 360 → MongoDB	KYC → MongoDB	Hồ sơ đính kèm → MongoDB	Hồ sơ claim → MongoDB

Lưu ý compliance: Ngành Ngân hàng (Basel II/III, Thông tư NHNN) và Thuế thường BẮT BUỘC audit trail đầy đủ — nhiều tổ chức vẫn ưu tiên Oracle/PostgreSQL cho MỌI dữ liệu core dù NoSQL kỹ thuật phù hợp hơn, đây là ràng buộc PHÁP LÝ cần tham vấn compliance trước khi chọn NoSQL. Dữ liệu KYC/cá nhân cần tuân thủ Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân — cân nhắc vị trí lưu trữ khi thiết kế.

PHẦN 8: CHECKLIST QUYẾT ĐỊNH & TỔNG KẾT

CHECKLIST NHANH – Chọn Database cho bài toán cụ thể:

- | | |
|--|------------------|
| ☐ Dữ liệu liên quan trực tiếp TIỀN/hợp đồng pháp lý? | → RDBMS bắt buộc |
| ☐ Yêu cầu compliance/audit trail nghiêm ngặt (bank/thuế)? | → Ưu tiên RDBMS |
| ☐ Schema thay đổi thường xuyên theo sản phẩm mới? | → MongoDB |
| ☐ Ghi vượt vài trăm nghìn record/giây, cần scale ngang? | → Cassandra/HDFS |

- | | |
|---|-------------|
| ☐ Trọng tâm tìm quan hệ nhiều bước giữa thực thể? | → Neo4j |
| ☐ Cần tốc độ dưới mili-giây cho dữ liệu tạm thời? | → Redis |
| ☐ Lưu trữ/phân tích PB dữ liệu thô, batch processing? | → Hadoop |
| ☐ KHÔNG rơi vào trường hợp đặc biệt nào?
(toàn, kiểm chứng) | → RDBMS (an |
-

Nguyên tắc cốt lõi:

- KHÔNG có "database tốt nhất tuyệt đối" – chỉ có "phù hợp nhất với TỪNG bài toán". Doanh nghiệp lớn LUÔN vận hành Polyglot Persistence, không bao giờ "một database cho tất cả"
 - Migration KHÔNG BAO GIỜ chỉ là "chuyển đổi cú pháp" – đặc biệt giữa RDBMS ↔ NoSQL, cần THIẾT KẾ LẠI mô hình dữ liệu theo triết lý hệ đích
 - Với Ngân hàng/Thuế, ràng buộc PHÁP LÝ/compliance đôi khi quan trọng hơn tối ưu kỹ thuật thuần túy
 - Mọi migration production cần: Assessment → Convert có kiểm soát → Dual-run validate → Cutover có kế hoạch rollback – không "big-bang" không có phương án lùi
-

Tài liệu tham khảo

- Oracle Database Documentation, ora2pg Official Documentation (ora2pg.darold.net)
- Microsoft SQL Server Migration Assistant (SSMA) for Oracle Documentation
- MongoDB Data Modeling Design Patterns, Neo4j Graph Data Modeling Guide
- Apache Cassandra/Kafka/Hadoop Official Documentation
- Nghị định 13/2023/NĐ-CP về Bảo vệ dữ liệu cá nhân

Bản quyền thuộc về Trần Văn Bình — Chuyên gia hàng đầu về System/Database tại Việt Nam

Hotline/Zalo: 090.29.12.888 · Website: www.tranvanbinh.vn